

STORMING AURORA HACKATHON

Participant Handbook

Everything you need to get from zero to a competing snake: the challenge, the build, the deploy, and the tournament.

Battlesnake · Autonomous Multiplayer Snake

Read once at kickoff · You shouldn't need anything else



CONTENTS

-
- 01 What the challenge is
 - 02 How the game actually works
 - 03 What you actually have to build
 - 04 Getting started, in your language of choice
 - 05 Deploying your snake for free
 - 06 Registering your snake
 - 07 The tournament & the day
 - 08 Fair play & practical tips
-

What the challenge is

You're building an autonomous **Battlesnake** , a program that plays a multiplayer snake game by itself. No manual control once the match starts: your code receives the board state each turn and decides where to move. Grow by eating, avoid colliding with any snake's body (yours or an opponent's), outlast the board.

You can use **any language and any approach** , hand-written heuristics, search algorithms, a trained model, or AI-assisted code. All of that is fair game and expected; this challenge is about how well your bot performs, not how it was written.

How the game actually works

- **Board:** a grid (11×11 for most of the event; a larger 19×19 board for the grand final specifically).
- **Each turn**, every snake picks one move: up, down, left, or right.
- **Food** spawns on the board; eating it grows your snake and restores health.
- **You die** if your head moves into any snake's body (including your own) or off the edge of the board, or if your health runs out.
- **You win** by being the last snake alive, or by having the best result when time runs out.

Ruleset changes by stage

Qualifying is **Standard** (the rules above). The bracket stage and grand final switch to **Royale**: the board's edge is a storm that closes in turn by turn and deals damage, so matches speed up and the fight always gets pushed into a shrinking space.

The grand final also plays on the biggest board, 19×19, more room, a longer and more dramatic finish.

A separate optional **constrictor** side event (a board mostly covered in hazard) may run after the final; it's announced on its own as a standalone bonus, outside the main tournament.

TIMING

Your server has **500 milliseconds** to respond with a move, every turn. Design for speed as much as for smarts.

What you actually have to build

A Battlesnake is just a small HTTP server. It needs to implement four endpoints:

GET `/` , Returns your snake's info: display name, color, head/tail style. Called periodically as a health check.

POST `/start` , Called once when a game begins. Use it to reset any per-game state your code keeps.

POST `/move` , The core of your bot. Called every turn with the full board state. Respond with `"up"` , `"down"` , `"left"` , or `"right"` .

POST `/end` , Called once when a game ends. Optional cleanup/logging.

That's the entire contract. Everything else , how you decide your move , is entirely up to you.

Getting started, in your language of choice

Battlesnake maintains official starter templates that already implement the four endpoints correctly, so you're writing decision logic on day one instead of wiring up a server from scratch:

- Python , `starter-snake-python`
- Go , `starter-snake-go`
- JavaScript / TypeScript , `starter-snake-node` / `starter-snake-typescript`
- Rust , community starter templates are available and well-maintained
- Ruby , `starter-snake-ruby`
- Other languages , search "battlesnake starter [language]"; the four-endpoint contract is simple enough to implement from scratch if needed.

Workflow

- 1 Clone the starter for your language.
- 2 Run it locally and confirm it responds correctly.
- 3 Edit the move-decision logic , this is the actual challenge.
- 4 Once it behaves sensibly, move on to Section 5 to get it a public URL.

Deploying your snake for free

Your snake needs a **public URL** so it can be registered and actually play matches. Here's what's genuinely free right now:

OPTION	BEST FOR	FREE?	CARD NEEDED?	SLEEPS?
Render	Any language (native or Docker)	Yes	No	Yes , 15 min idle, ~30–60s wake
Google Cloud Run	Any language via Docker	Yes, generous quota	Usually	Yes , scales to zero
PythonAnywhere	Python specifically	Yes	No	No , stays warm
Netlify / Vercel	JS/TS (Python on Vercel)	Yes	No	Yes , serverless cold start
Self-host + tunnel	Any language , the universal fallback	Yes	No	No (if laptop stays on)

CRITICAL

Most free options "sleep" after inactivity, and waking can take longer than the 500ms move limit. **Send your snake a request a few minutes before your match** to keep it awake. This is the single most common way a good bot loses a match it should have won.

Rust or C++?

Render and Google Cloud Run both work well via Docker. Self-hosting with a tunnel is also genuinely good , it sidesteps language support questions entirely, at the cost of needing your laptop running.

Registering your snake

- 1 Create a free Battlesnake account.
- 2 Add your snake: a name, your deployed URL, and optionally custom colors/head/tail.
- 3 Test it , send it into a practice match against itself to confirm it responds correctly.
- 4 Submit your registered snake's details through the event's tournament page , details shared at kickoff.

Keep your URL stable through the day. If you redeploy and your URL changes, update your registration immediately so you're not unreachable for a match.

How the tournament actually works

Qualifying

Everyone plays several matches in **Standard** mode, 4 snakes per board, opponents reshuffled each round. Placement points (1st = 10, 2nd = 6, 3rd = 3, 4th = 1) are summed over all your matches to build a leaderboard.

Bracket stage

Top finishers from qualifying are seeded into a single-elimination **Royale** bracket. The storm shrinks the board turn by turn and deals damage, matches stay fast and someone always gets cornered. Win to advance, lose once and you're out.

Grand final

The last two survivors face off on the big **19×19 Royale** board, live and projected for everyone to watch. The winner is the champion.

Side event (optional)

A standalone **constrictor** bonus round on a mostly-hazard board, with its own leaderboard. It doesn't affect who wins the main tournament.

The rules are never a surprise: Standard while qualifying, Royale from the bracket onward. The only board-size change is the grand final (11×11 → 19×19).

The day, start to finish

- **Kickoff**, this handbook goes out, along with the tournament page/registration link and any final rule clarifications.
- **Build window (12 hours)**, develop, test locally, deploy, register your snake, and iterate. An unscored practice window partway through the day is a good moment to test against real opponents.
- **Code freeze**, have your snake deployed, awake, and registered well before this. Don't be mid-deploy when matches start.
- **Live tournament**, qualifying → bracket → grand final (followed by the optional side event), run and projected live.
- **Results**, winners announced at the end of the grand final.

Fair play

- AI-assisted coding is explicitly allowed and expected , use it.
- No interfering with another team's server or connection.
- No exploiting engine bugs; play within the rules as described.
- Be ready to show your source code if asked , this is about verification and mentor support, not suspicion.

A few practical tips

- Your snake may be playing more than one game at a time during heats , make sure your server handles concurrent requests.
- Keep your move logic fast , every millisecond counts against that 500ms limit, especially with search or simulation.
- Test against multiple opponents at once, not just a single practice bot , 4-snake matches behave differently from 1v1.
- Re-check that your deployed URL is reachable from outside your own machine , "works on localhost" isn't the same as "works when the tournament calls it."